LOW-LEVEL DESIGN

RAG-Based Customer Support Assistant with LangGraph & HITL

Document	Low-Level Design — RAG Customer Support Assistant
Intern	Zaara Shaikh | ID: IN226050502
Org	Innomatics Research Labs — Advanced GenAI Internship Feb 2026
Date	April 23, 2026

1. Module-Level Design

1.1 Document Processing — load_and_chunk() [rag_pipeline.py]

Reads PDF via PyPDFLoader (langchain-community). Returns List[Document] with page_content and metadata.

```
from langchain_community.document_loaders import PyPDFLoader Input: str (pdf_path)
Output: List[Document]
```

1.2 Chunking — RecursiveCharacterTextSplitter [rag_pipeline.py]

Splits Document list into ~500-token chunks with 50-token overlap. Uses langchain_text_splitters (updated modular package, replacing deprecated langchain.text_splitter).

```
from langchain_text_splitters import RecursiveCharacterTextSplitter chunk_size=500,
overlap=50 separators=['\n\n','\n','.', ' ']
```

1.3 Embedding — build_vectorstore() [rag_pipeline.py]

Wraps HuggingFaceEmbeddings with all-MiniLM-L6-v2 from sentence-transformers. Runs on CPU — no GPU or API key required. Produces 384-dim float vectors.

```
from langchain_community.embeddings import HuggingFaceEmbeddings
model_name='all-MiniLM-L6-v2', device='cpu' Output dim: 384
```

1.4 Vector Storage — build_vectorstore() / load_vectorstore() [rag_pipeline.py]

ChromaDB via langchain-chroma package (updated from deprecated langchain_community.vectorstores.Chroma). Auto-persists to ./chroma_store — no explicit .persist() call needed in current version.

```
from langchain_chroma import Chroma persist_directory='./chroma_store'
collection_name='support_docs'
```

1.5 Retrieval — get_retriever() [rag_pipeline.py]

LangChain retriever wrapping ChromaDB. Returns top k=4 chunks by cosine similarity. Query is embedded on-the-fly using the same model as ingestion.

```
retriever = vectorstore.as_retriever(search_kwargs={'k': 4})
```

1.6 Query Processing — generate_node() [graph_workflow.py]

Builds context+query prompt, sends to llama-3.1-8b-instant via ChatGroq. Temperature=0.2 for factual, low-hallucination output. Model switched from llama3-8b-8192 which was decommissioned by Groq's API during testing.

```
from langchain_groq import ChatGroq model='llama-3.1-8b-instant', temperature=0.2
Prompt: answer ONLY from context; admit uncertainty
```

1.7 Graph Execution — build_graph() [graph_workflow.py]

Assembles LangGraph StateGraph with 5 nodes: retrieve, generate, route, hitl, output. Entry point: retrieve. Conditional edge at route dispatches to hitl or output based on needs_human.

```
from langgraph.graph import StateGraph, END StateGraph(SupportState)
add_conditional_edges('route', should_escalate, {'hitl':'hitl','output':'output'})
```

1.8 HITL Module — hitl_node() [graph_workflow.py]

Activated when route_node sets needs_human=True. Prints escalation banner, customer query, and AI draft. Accepts human agent input via stdin. EOFError caught for non-interactive environments (automated tests). Testing confirmed: 'lawsuit' query correctly triggers this node.

```
Trigger: needs_human==True Input: human override (or Enter to accept draft) Output:
final_output with [Reviewed by Human Agent] prefix
```

1.9 Environment Setup

The project runs in a Python virtual environment (.venv). GROQ_API_KEY is set as an environment variable — not hardcoded. sentence-transformers is required for local CPU embedding (HuggingFaceEmbeddings). All packages are pinned in requirements.txt.

```
pip install langchain langchain-community langchain-text-splitters langchain-chroma
langchain-groq langgraph chromadb sentence-transformers pypdf set
GROQ_API_KEY=your_key_here # Windows export GROQ_API_KEY=... # macOS/Linux
```

2. Data Structures

2.1 LangChain Document Object

```
Document(page_content: str, metadata: dict) # {'source': 'faq.pdf', 'page': 3}
```

2.2 Chunk Format

After splitting, each chunk is still a LangChain Document object with page_content (the chunk text, max ~500 tokens) and metadata inherited from the parent page: {"source": "faq.pdf", "page": 2}. The chunk text is plain UTF-8 string. No special formatting is applied before embedding.

2.3 Embedding Structure

```
chunk_embedding: List[float] # length 384 (all-MiniLM-L6-v2) query_embedding:
List[float] # same vector space, computed at query time similarity: float # cosine
similarity [0.0, 1.0]
```

2.4 SupportState TypedDict — LangGraph State Object

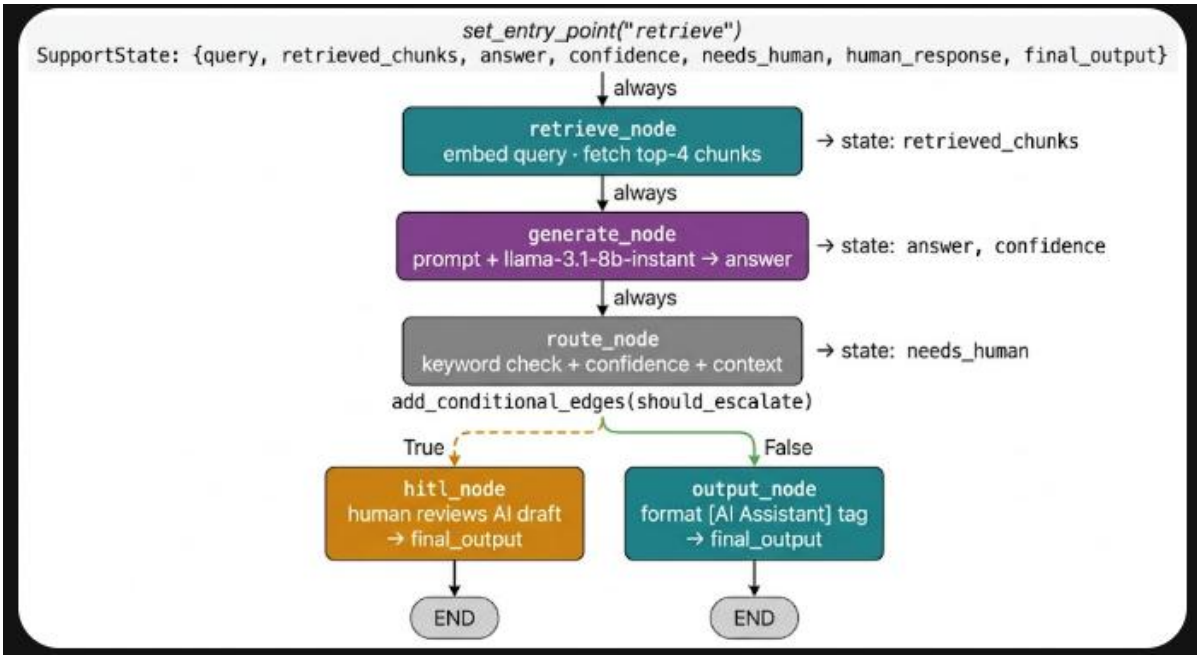
```
class SupportState(TypedDict): query: str # original user question retrieved_chunks:
list[str] # text of top-4 retrieved docs answer: str # LLM-generated answer confidence:
str # 'high' | 'low' needs_human: bool # escalation flag (set by route_node)
human_response: Optional[str] # agent override input final_output: str # shown to user
```

2.5 Query-Response Schema

CLI: plain text in/out. Prefix: [AI Assistant] or [Reviewed by Human Agent]. Future API: POST /query → {query, answer, source, confidence, escalated} as JSON.

3. LangGraph Workflow Design

Full state machine diagram. SupportState flows through every node. The conditional edge at route_node dynamically selects the next node based on the needs_human flag set by the routing logic.



Edges:

- **retrieve → generate (unconditional)**
- **generate → route (unconditional)**
- **route → hitl (conditional: needs_human == True)**
- **route → output (conditional: needs_human == False)**
- **hitl → END**
- **output → END**

Node	Function	Reads from state	Writes to state
retrieve	retrieve_node()	query	retrieved_chunks
generate	generate_node()	query, retrieved_chunks	answer, confidence
route	route_node()	answer, confidence, query	needs_human
hitl	hitl_node()	query, answer	human_response, final_output
output	output_node()	answer	final_output

4. Conditional Routing Logic

4.1 Answer Generation Criteria

The generate_node always attempts to produce an answer as long as the graph reaches it. Even if retrieved_chunks is empty, the LLM still runs — but with an empty context, it will produce a low-confidence response (the prompt explicitly says "if the answer is not in the context, say clearly that you don't have that information"). This empty-context case is detected downstream in route_node via len(retrieved_chunks) == 0 which directly triggers escalation.

4.2 Escalation Criteria

Condition	Detection Method	Confirmed Test Case
Low confidence	Keywords in answer: "cannot", "not sure", "no inform	"I don't know", "I don't have information about that"
Complex / sensitive query	Keywords in query: "lawsuit", "fraud", "legal", "billing dispute", "to file a lawsuit", "lawsuit against"	"I want to file a lawsuit against Apple" → escalated (c onfirmed in
No context found	len(retrieved_chunks) == 0	Query about topic not in PDF → no chunks → HITL

4.3 Routing Code

```
def should_escalate(state: SupportState) -> str: return 'hitl' if state['needs_human']
else 'output' # In route_node: is_complex = any(kw in query_lower for kw in
COMPLEX_KEYWORDS) is_low_conf = state['confidence'] == 'low' no_context =
len(state['retrieved_chunks']) == 0 needs_human = is_complex or is_low_conf or
no_context
```

5 HITL Design

5.2 Confirmed Working Behaviour (from testing)

Testing confirmed the HITL flow end-to-end:

5.2.1 Query: "How do I delete a contact?" → retrieve_node gets 4 chunks → generate high confidence → route to direct output

5.2.2 Query: "I want to file a lawsuit against Apple" → route_node detects "lawsuit" → escalation banner printed → human agent input prompted

5.2.3 Human agent can type a corrected response or press Enter to accept the AI draft

5.2.4 [Reviewed by Human Agent] tag prepended to final_output on escalation path

5.3 What happens after escalation

After escalation triggers, the system prints the customer's original query and the AI draft answer. The human agent either types a corrected response or presses Enter to accept the draft. The final response is stored in human_response in the state object and returned with a [Reviewed by Human Agent] prefix. The graph then terminates at END.

5.4 Production Integration Path

In production: LangGraph's interrupt() mechanism pauses the graph at hitl_node. Escalated query + AI draft are pushed to a ticketing system (Zendesk/Freshdesk). The graph resumes when the agent submits a response via API callback. State between pause and resume is persisted in PostgreSQL.

6 API / Interface Design

6.2 Current CLI Interface

```
python main.py --ingest --pdf # Ingestion mode python main.py --query # Interactive mode
python main.py --ask "" # Single query
```

6.3 Future FastAPI Interface

```
POST /query Request: { "query": "What is your return policy?" } Response: { "answer":
"...", "source": "ai"|"human", "confidence": "high"|"low", "escalated": false }
```

7 Error Handling

Error	Where	Handling
PDF not found	load_and_chunk()	FileNotFoundError raised with path in message
No chunks retrieved	retrieve_node()	Empty list → route_node sets no_context=True → HITL
Model decommissioned (Groq 400)	get_llm()	Caught during testing. Resolved by switching model to llama-3.1-8b-inst

t in get_llm()

chain_chrom